

Introduzione al C

Lezione 9

Sistemi Operativi - II Modulo (Canale A - L)

Prof. Paolo Zuliani

Dipartimento di Informatica

(Materiale didattico per cortesia del Prof. Emiliano Casalicchio)



SAPIENZA
UNIVERSITÀ DI ROMA



Agenda

- Leggere la linea di comando
- Allocazione dinamica della memoria
- Strutture
- Input/Output da file



Leggere la linea di comando



Linea di comando

- `main` ha due parametri, convenzionalmente chiamati `argc` e `argv`

```
int argc           // il numero di 'parole'  
char *argv[]       // le 'parole' digitate
```

Ad esempio, digitando:

```
$: cat file.txt
```

Avremo che:

```
argc = 2  
argv[0] = "cat"  
argv[1] = "file.txt"
```



Linea di comando: Esercizio

Completare il seguente codice in modo tale che il programma stampi tutte le parole della sua linea di comando.

```
int main(int argc, char *argv[]) {  
    while (??? >= 0)  
        printf("argv[%d] = %s\n", ???, argv[???]);  
    return 0;  
}
```

Linea di comando: Soluzione

```
int main(int argc, char *argv[]) {  
    while (--argc >= 0)  
        printf("argv[%d] = %s\n", argc, argv[argc]);  
    return 0;  
}
```

Allocazione dinamica della memoria



I puntatori

- Abbiamo finora visto cose tipo:

- `char *strcpy(char *dest, const char *src, size_t n);`
- `scanf("` `...` `", &varname);`

- Hanno a che fare con i **puntatori**, ossia variabili che contengono **l'indirizzo di un'altra variabile (i.e., una locazione di memoria)**

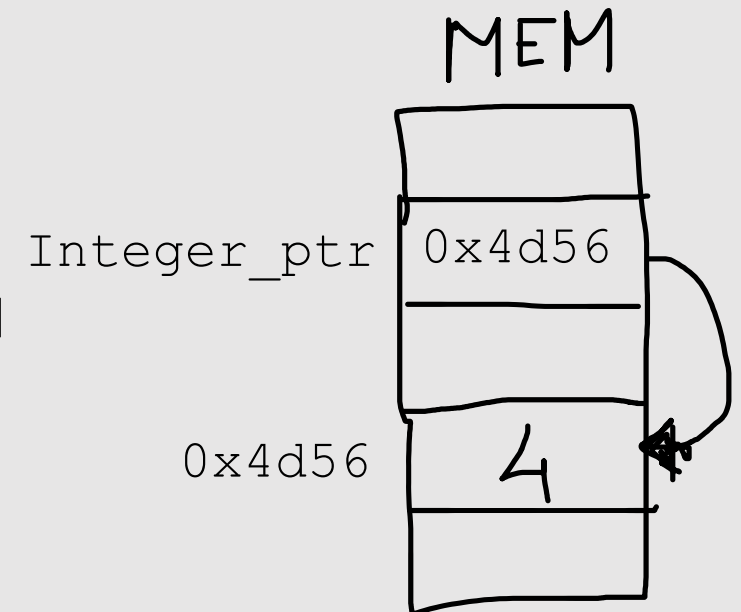
- `<tipo> *var_name;`

- Ad es:

- `char *dest;` `// puntatore ad un carattere`
- `int *intPointer;` `// puntatore ad un intero`

Valori associati ad una variabile puntatore

- `int *integer_ptr;`
- Valore *diretto*
 - Indirizzo di un'altra cella di memoria (ad es: 0x4d56)
 - Vi si accede mediante il nome della variabile (ad es: `integer_ptr`)
- Valore *indiretto*
 - Valore contenuto dalle cella di memoria in cui indirizzo è il valore diretto
 - Vi si accede mediante, ad es:, `*integer_ptr`
- Ad es:
 - Valore diretto = 0x4d56
 - Valore indiretto = 4



Operatori & e *

```
int num=5;  
int *numPtr;
```

- `numPtr = #`
 - assegna a `numPtr` l'indirizzo di `num`
- `*numPtr = 10;`
 - assegna alla *locazione di memoria* puntata da `numPtr` il valore 10
- Vedi il file `pointer.c`

Aritmetica dei puntatori

- Un puntatore contiene un numero intero: è l'*indirizzo* di una cella di memoria.
- Possiamo quindi applicare alcune operazioni aritmetiche, ad es.:

```
int *ptr;
```

```
ptr = ptr + 1;    //ptr viene incrementato di 4 bytes  
                  (sizeof(int)) (locazioni di memoria)
```

```
ptr = ptr + n;    //ptr viene incrementato di 4n bytes  
                  (locazioni di memoria)
```

Vedi `aritmeticaPtr.c`



Vettori (array) e puntatori

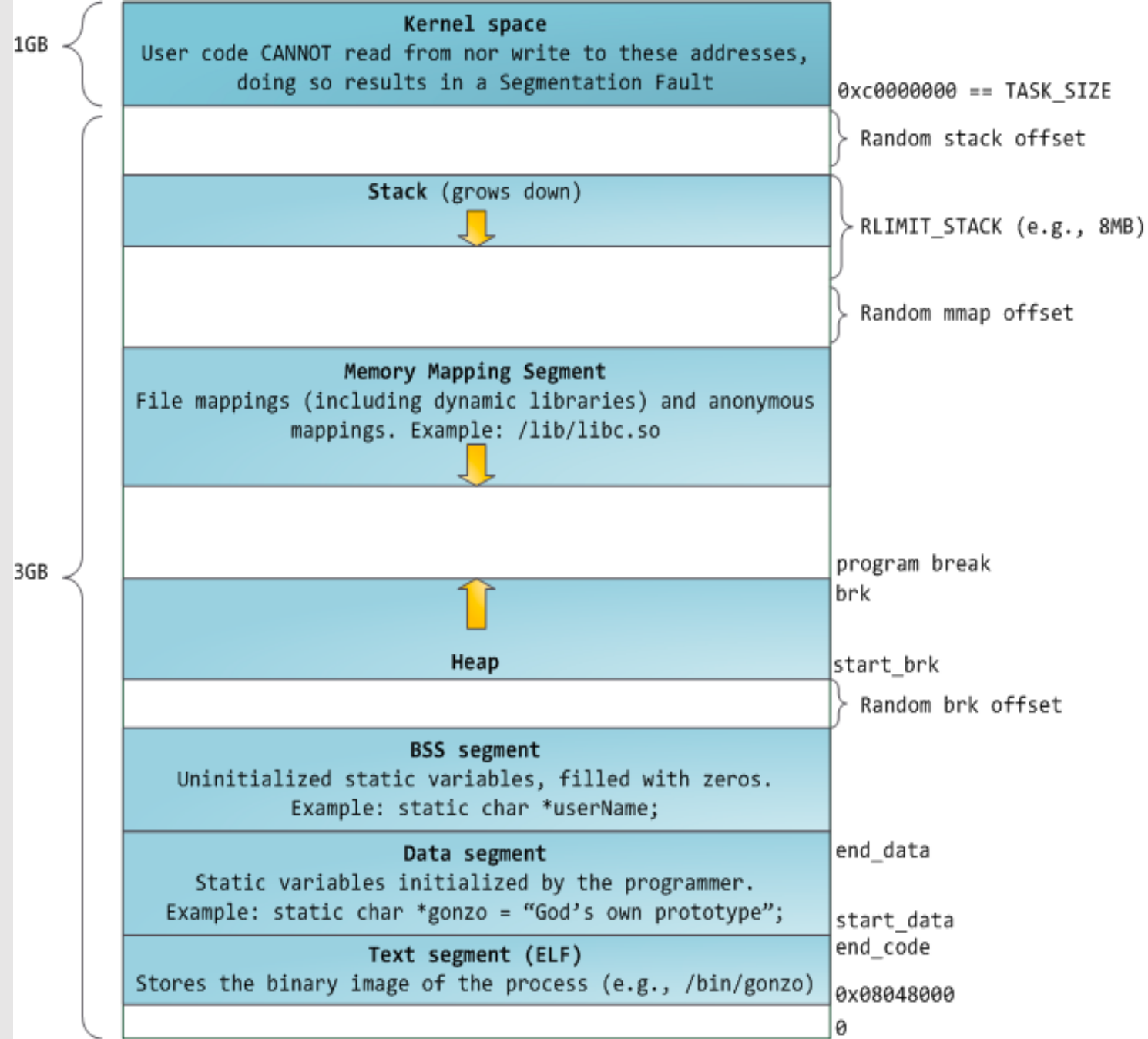
```
int vect[10];  
int *ptr = NULL; //puntatore 'nullo'  
ptr = &vect[0]; //puntatore al primo elemento  
ptr = vect;      //puntatore al vettore
```

Puntatore al primo elemento e variabile vettore/array sono la **stessa cosa!!**

Vedi `arrayPtr.c`

Allocazione dinamica

- Vettori e altre variabili sono allocate nello **stack** a tempo di *compilazione*
- La memoria richiesta a tempo di *esecuzione* viene allocata nello **heap**



Allocazione dinamica

- La memoria richiesta a tempo di *esecuzione* viene allocata nello **heap**
- Vediamo come allocare memoria a tempo di *esecuzione*

```
void *calloc(size_t nmemb, size_t size);  
void *malloc(size_t size);  
void free(void *ptr);
```

Vedi heapAlloc.c



Allocazione dinamica

1. Si richiede memoria dinamica tramite `calloc` o `malloc`
 2. Dopo l'uso rilasciare la memoria con `free`
- Il programmatore C deve farsi carico del *garbage collection* ('raccolta della spazzatura', i.e., liberare memoria dinamica non più in uso)
 - Il parametro di `free` **deve** essere un puntatore ritornato da `calloc/malloc`, altrimenti il comportamento di `free` è **indefinito**
 - Chiamare `free` più di una volta con lo stesso puntatore è un uso **indefinito**

Esercizio 5

Leggete **ATTENTAMENTE** il man per le differenze tra
`calloc` e `malloc`

- Implementare l'esempio `heapAlloc.c` usando `malloc`

Esercizio 6 (a casa)

- Progettare ed implementare un programma che
 - Allochi dinamicamente 2 blocchi di memoria, chiamiamoli `m1` e `m2`. I due blocchi hanno dimensione arbitraria (ad esempio letta come argomento di `main`)
 - Inizializzi il blocco `m1` con dei dati, ad esempio leggendoli da un file di input (passato da `stdin`)
 - Copi il blocco di memoria `m1` in `m2` (usare `memcpy()`)
 - Devono essere fatti gli opportuni controlli sulle dimensioni dei 2 blocchi di memoria, ad esempio se `m2` è più piccolo copiare solo parzialmente `m1`.
 - Produca come output il contenuto di `m2` (ridirezionare `stdout` su di un file di testo)

Strutture



Strutture (structures)

- Definizione di nuovi tipi di dato che combinano tipi di dato eterogenei
 - Possiamo vederla come un contenitore di una collezione di variabili di vari tipi
 - Permette di trattare tale collezione come un tipo di dato da noi definito
- Devono essere definite prima di essere usate (ovvio!!!)
- Tre modi per definirle
 - Variabile struttura (*variable*)
 - Struttura etichettata (*tagged*)
 - Definizione di tipo struttura (*type-defined*)



Variabile struttura

```
struct { // x,y: nomi dei membri della struttura
    double x; // coordinata x
    double y; // coordinata y
} point2D; // point2D: nome variabile
```

- Poco portabile, tutte le variabili struttura vanno definite qui:

```
struct {
    ...
} pA, pB, pC; // point2D: nome variabile
```

Tagged structure

```
struct point3D {  
    // point3D: tag  
    // x,y,z: nomi dei membri della struttura  
    double x; // coordinata x  
    double y; // coordinata y  
    double z; // coordinata z  
};
```

```
struct point3D pointA, pointB, pointC;
```

- Soluzione più portabile, potrei mettere la definizione di `struct point3D` in un header file e riutilizzarla



Type-defined structure

```
typedef struct {  
    char ID[17]; //ad es: codice fiscale  
    long int income;  
    float taxRate;  
} taxpayer_t; //nome del nuovo tipo di dato
```

```
taxpayer_t person1, person2;  
taxpayer_t persons[100];
```

Inizializzazione strutture

```
struct point3D pointA={1.1, 1.2, 3.5},  
                    pointB={0.3, 4.5};
```

- Inizializzare meno campi è legale

Allocazione dinamica di strutture

- Vedi `taxPayersAlloc.c`



Vediamo `taxPayers.c`

- **Esercizio 1.** in `taxPayers.c` modificare la funzione `newTP` in modo che prenda come parametro di input anche la data di nascita come tipo di dato `birthdate_t` (fatelo subito 10min)
- **Esercizio 2.** in `taxPayers.c` modificare `main` in modo che i dati del contribuente siano inseriti da linea di comando (uso di `argc` e `argv`)
- **Esercizio 3.** in `taxPayers.c` modificare il codice in modo che i dati del contribuente siano letti da file di testo (uso ridirezione `stdin`)

Passaggio delle strutture come parametri a funzione

- Per valore
 - Vantaggio: più semplice/intuitivo
 - Svantaggio: dati vengono copiati nello stack
- Per riferimento (puntatore)
 - Vantaggio: i parametri non vengono memorizzati nello stack andando così ad ottimizzarne l'uso essendo una risorsa limitata
- Vedi `taxPayersParams.c`

File Input/Output



Tipi di file

- Testo
- Binari
- Buffer
 - Area di memorizzazione temporanea che mantiene i dati non ancora trasferiti dal disco alla memoria o viceversa



File di testo

- Tutte le funzioni per la loro gestione sono in `stdio.h`
- Ogni linea (riga) finisce con `\n`
- Il file finisce con `EOF` (definito in `stdio.h`)

Passi per usare i file

vedi `file.c`

1. Dichiarare un *file pointer*
2. Aprire il file
 - Viene creata una struttura dati con le info necessarie per gestire il file
 - Viene allocato l'area di buffer
 - Collega il *file pointer* con la struttura
3. Usare le funzioni di I/O
 - Processo (progr. in exec.) → buffer → disco (e viceversa)
4. Chiudere il file
 - Scrive il contenuto buffer nel file (se necessario)
 - Libera la memoria associata al file pointer

Input/output

- `int fscanf(FILE *stream, const char *format, ...);`
- `int fprintf(FILE *stream, const char *format, ...);`
- `char *fgets(char *s, int size, FILE *stream);`
 - legge **size** caratteri o fino a `\n`
- `int fputs(const char *s, FILE *stream);`
- `int feof(FILE *stream);`
 - restituisce un valore diverso da 0 quando ha trovato EOF

fscanf/fprintf

- vedi `filefsfpf.c`
- Richiedono la conoscenza della struttura del file

fgets

- Vedi `filefgets.c`
- Esercizio 4
 - Modificare `filefgets.c` in modo che il programma legga tutto il contenuto del file, fino alla fine
 - Suggerimento: leggete il man di `fgets` (specialmente la parte sul valore di ritorno)!

fputs

- Vedi `filefputs.c`
- Esercizio 5
 - scrivere un programma che legge un intero file e lo riscrive in altro file
- Esercizio 6
 - considerate l'esercizio 3. Modificate il programma in modo che i dati del tax payer vengano letti da un file di testo usando sia `fscanf` sia `fgets`

Domande?

